LINGLONG SG98

How To Add A New Motor Backend

A simple, low-risk guide for bringing a new RS-485 motor family into the shared motor architecture.

Copy RT, make probe safe, register once.

Source guide: Documentation/New-Motor-Backend-Guide.md

Know The Split Before You Edit

The shared layer picks the drive and exposes common controls. The backend owns every motor-specific detail.

Shared layer

src/motor_hal.* owns DriveType, command tables, motor descriptors, and AUTO detect selection.

Backend

Each family implements probe, init, tick, off, calibrate, sleep, and optional reset in its own motorcontrol_*.cpp.

Starting point

Use RT for a straightforward register drive. Use JMC when you need multi-baud probing or a more stateful protocol.

Key files: src/motor_backend.h, src/motor_hal.h, src/motor_hal.cpp

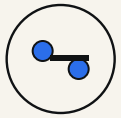
Collect The Manual Facts First

Do not start coding until you know the safe registers and the bus rules.



Bus settings

Baud, parity, stop bits, slave ID, and whether the drive boots with a predictable default profile.



Control path

Run, stop, direction, torque, RPM, save, reset, and which writes are safe during bring-up.



Telemetry

A read-only status block, actual RPM, actual torque or load, and the fault register you can trust.

Bring-up inputs: bus defaults, safe probe reads, run/stop path, RPM, torque, fault

Create The Backend And Register It

Most bring-up work is wiring the new family into the shared tables cleanly.

Create files

Copy src/RT/motorcontrol_rt.h/.cpp into src/FOO/, rename the symbols, and keep drive logic inside those files.

Add type

Extend DriveType in src/motor_hal.h and add only the MotorProbeReport fields you need.

Register

Include the new header in src/motor_hal.cpp, add a FOO command table, then add a MotorDescriptor entry.

Edit: src/FOO/motorcontrol_foo.h/.cpp and src/motor_hal.h/.cpp

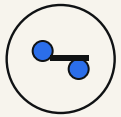
Make The Probe Correct Before Runtime

A weak probe breaks every motor, not just the new one.



Switch first

Use motor_probe_set_profile(...) before probing so the bus is in the right timing and serial mode.



Match precisely

Read a register that is distinctive to the new family. Return true only on a repeatable, high-confidence response.



Stay safe

Avoid destructive writes during probe. If you need multi-baud probing, follow the JMC pattern and restore the preferred profile.

Use: src/motor_probe.h, src/motor_probe.cpp, and the JMC multi-baud probe pattern

Fill The Standard Hooks And Data Mapping

Once the probe is solid, the rest of the system already knows how to use the backend.

tick()

Read status, update motor_currentRPM and motor_currentTorque, apply motor_setRPM, and keep currentStatus honest.

Menus

Use MotorCommandDescriptor for menuNum, itemIdx, valueScale, and supported. Most motors need table changes, not UI changes.

Comm profile

Only add a new CommProfile when baud, parity, or timing truly differ. Keep Serial2 setup in comm.cpp.

Map commands in motor_hal.cpp. Keep serial behavior in src/comm.h and src/comm.cpp

Bench-Test In This Order

Keep the system debuggable and verify the basics before tuning.

1. Detect + stop

Confirm AUTO detect picks the right drive, forced selection works, and off() stops cleanly from idle and while running.

2. Telemetry + direction

Check positive RPM, reversed direction, actual RPM, and torque or load values against what the motor is really doing.

3. Sleep + recovery

Verify unplug or power-cycle recovery, sleep behavior, reset behavior, and that unsupported menu items stay unavailable.

Bench test on real hardware before tuning torque, ramp, save, or reboot flows

What To Put In The Pull Request

A good backend is small, explicit, and easy for the next person to trust.

Required files

Backend .h/.cpp, DriveType, optional probe fields, command table, descriptor entry, and comm profile changes only when required.

Bench notes

Name the manual, the key probe and status registers, what was tested on hardware, and anything not yet verified.

Rule of thumb

If you are unsure where code belongs, keep the shared layer generic and push motor-specific behavior down into the backend.